Co-Scheduled Adjoint Greeks in a Fixed-Point FPGA Datapath: Heston-Model Option Sensitivities with First-Order Rounding Bounds

Yash Agrawal
Birla Institute of Technology and Science, Pilani
Pilani, India

Govind Prasad
Birla Institute of Technology and Science, Pilani
Pilani, India

***Abstract*—FPGA option-pricing engines obtain Greeks, the sensitivities of a price to its inputs, by bump-and-reprice or by re-simulation. We present an FPGA engine for European options under the Heston model, priced with the COS Fourier-cosine method, in which a hand-derived reverse (adjoint) sweep is co-scheduled with the forward pass in one statically scheduled fixed-point datapath. Because the COS loop has no data-dependent control flow, the values the reverse sweep needs are held in a statically allocated, register-resident tape rather than in memory. A generator emits, from one operation graph, a bit-accurate model, the modulo-scheduled Verilog and a first-order rounding-error bound for the price and each of nine first-order Greeks, used at design time to size and certify the word length. Implemented in Vivado on a Zynq-7020 at equal throughput, the price-and-Greeks engine takes 1.03× the cycles of a price-only engine at 1.68× its LUTs and 2× its DSP blocks; a factored forward-mode engine built by the same flow needs 7% more LUTs, 12% more flip-flops and 25% more DSP blocks and is 6× less accurate at the same word length. The bound was never exceeded over 21,000 inputs, and the engine ran bit-exact on two FPGA families (1,469/1,469 outputs, including 50 calibrations to NIFTY options). The bound covers rounding only; the COS truncation error dominates in a few in-domain cases. The computed engine latency on a ZCU102, 15.9 μs excluding bus transfers, is 4.9–15× below general-purpose AD and bump-and-reprice on a laptop core but not below a vectorised implementation of hand-derived Greeks (13.6 μs).**

***Index Terms*—FPGA, adjoint differentiation, option Greeks, Heston model, COS method, fixed-point arithmetic, rounding-error analysis, modulo scheduling**

I. Introduction

Option Greeks, the partial derivatives of a price with respect to spot, strike, maturity, rate and the model's parameters, set hedge ratios and risk limits and drive model calibration. Under the Heston model [1] the price has no closed form and is computed numerically, here by the COS method [2]. Published FPGA engines compute Greeks by bump-and-reprice or by re-simulation [14], [15], [21]: nine Greeks by central differences cost 19 pricings, with an accuracy that depends on a bump size no setting of which suits every Greek (Fig. 5). Software uses adjoint algorithmic differentiation (AAD) [5]–[7], which returns all derivatives of one output in a single reverse sweep but records a tape of the forward computation; our taped CPU baseline costs 7.6 pricings.

A reverse sweep over code without data-dependent control flow can be scheduled statically, as in source-transformation AD [10] and neural-network training accelerators. The COS pricer has this property: each of its 128 terms runs the same operations. We co-schedule a hand-derived reverse sweep with the forward pass in one fixed-point datapath and measure what the Greeks cost in time, area and accuracy against alternatives built through the same flow. Contributions:

1) **A co-scheduled adjoint datapath** for Heston–COS Greeks with a statically allocated, register-resident tape (Section III). After a literature search (Section II) we know of no prior FPGA computation of Greeks by an adjoint and no prior FPGA COS implementation.
2) **A measured cost of the Greeks**: price-only, adjoint and factored forward-mode engines of equal throughput implemented in Vivado through the same flow (Section V-A).
3) **A first-order rounding-error bound for every Greek** of a datapath that contains its own reverse sweep, extending AD-based precision analysis for FPGAs [25], [26], with its scope and the modelling of the primitives stated (Sections III, V-C).
4) **A generator and bit-exact silicon results** on two FPGA families, including calibrations to market data (Section V-D).

Not claimed: an AD tool (the reverse sweep is derived by hand); a bound attached to each result (it is computed at design time and covers rounding only); speed or energy advantages over a vectorised CPU (Section V-E).

II. Background and Related Work

Heston and COS. The Heston model has $\mathrm{d}S_t = rS_t\,\mathrm{d}t + \sqrt{v_t}S_t\,\mathrm{d}W_1$, $\mathrm{d}v_t = \kappa(\theta - v_t)\,\mathrm{d}t + \xi\sqrt{v_t}\,\mathrm{d}W_2$, $\mathrm{d}W_1\mathrm{d}W_2 = \rho\,\mathrm{d}t$ and nine pricing inputs $p = (S_0, K, T, r, v_0, \kappa, \theta, \xi, \rho)$; we use the characteristic function φ in the form of Albrecher et al. [4].

TABLE I
RELATED WORK (FIGURES NOT DIRECTLY COMPARABLE)

Work	Platform	Pricing	Greeks by
[17], [18]	FPGA	Heston MC	— (price only)
[19], [20]	FPGA	trees, quadrature	—
[14]–[16]	FPGA, ACAP	STAC-A2 (Heston MC)	re-simulation
[21]	FPGA (HLS)	Heston MC	central differences
[22], [23]	GPU	COS	— (price only)
[24]	GPU	general	taped adjoint AD
[12]	CPU	Heston, Fourier	analytic (by hand)
This work	FPGA	Heston, COS	**co-scheduled adjoint, per-Greek rounding bound**

The COS method [2] expands the density on a truncation range $[a, b]$ in a cosine series; for a put

$$V_{\text{put}} \approx Ke^{-rT} \sum_{k=0}^{N-1}{}' \operatorname{Re}\Big[\varphi(u_k)\, e^{\,iu_k(x-a)}\Big]\, U_k, \quad u_k = \tfrac{k\pi}{b-a}, \tag{1}$$

with $x = \ln(S_0/K)$ and closed-form payoff coefficients U_k; calls follow from put–call parity, and the N terms are independent. Delta and Gamma have closed-form COS expressions [2, Remark 3.2], [3], and $\partial V/\partial K$ follows from Delta, so these are nearly free in any COS implementation; the derivatives in T, r and the five model parameters enter through φ, and these are where differentiation does real work.

Greeks. Bump-and-reprice has truncation error $O(h^2)$ and rounding error $O(\epsilon/h)$. Forward-mode AD carries a tangent per input; reverse-mode AD returns all derivatives in one sweep at a small multiple of the pricing cost [8], storing forward values on a tape or recomputing them under checkpointing [9]. Cui et al. [12] derived the Heston–Fourier gradient by hand.

Hardware. FPGA option pricing covers Heston Monte Carlo [17], [18], trees [19] and quadrature [20]; the STAC-A2 risk benchmark, with Greeks by re-simulation, has been ported to Alveo, Stratix-10 and Versal devices [14]–[16], and AMD's library computes Greeks by central differences [21]. COS pricing has been accelerated on GPUs, including Heston and many strikes [22], [23], and adjoint AD runs on GPUs with a tape [24]. Table I summarises. We found no FPGA adjoint Greeks and no FPGA COS implementation in FPL, FCCM, FPT, ReConFig, TRETS, HEART, H2RC and general indexes; absence from a search is not proof of absence.

Precision analysis. AD-based word-length analysis for FPGAs goes back to Gaffar et al. [25], accuracy-guaranteed bit-width optimisation to Lee et al. [26], and Gappa [27] gives machine-checked enclosures. Our bound is Linnainmaa's first-order analysis [11] applied to a datapath that contains its own adjoint; we do not compare its tightness with affine-arithmetic or Gappa enclosures.

III. METHOD

A register-resident tape. A reverse sweep needs the forward values in reverse order. In (1) every term runs the same operations, the range is fixed per evaluation, and nothing branches on data, so the forward and reverse operations of one term form a fixed graph whose every value has a lifetime known at design time. The modulo scheduler allocates each value the rotating registers that hold it across the terms in flight: the tape costs registers, not memory traffic. Over all terms in flight the 56-bit adjoint datapath needs 447 such registers, against 150 for a price-only datapath of equal throughput and 374 for the factored forward one. The Greeks are derivatives of the COS approximation with $[a, b]$ held fixed (A4, Table III); per term the sweep propagates adjoints to the model inputs, reusing the forward reciprocals, a power-of-two normalisation keeps small adjoints at full precision, and nine accumulators sum the results.

Arithmetic. No dividers: reciprocals and square roots use Newton–Raphson from table seeds, the exponential Tang's method [28], the logarithm a table and series, and sines, cosines and complex magnitudes and angles CORDIC [29]. A term costs 250 multiplications, three CORDIC rotations and two vectorings (126 multiplications for the price alone). Values are fixed point with FL fraction bits (56-bit words with $FL = 28$, or 64 with $FL = 32$), rounded to nearest after every operation; shifters are built only as wide as the ranges measured over the domain, and an input beyond a range raises `range_err`.

First-order rounding-error bound. With local errors $|\delta_i| \le \epsilon_i$ at operations w_i, to first order

$$|\hat{y} - y| \;\le\; \sum_i |\partial y/\partial w_i|\, \epsilon_i =: B_y, \tag{2}$$

Linnainmaa's analysis [11]; the sensitivities come from a reverse sweep over the graph, which includes the hardware's own adjoint operations, so B_y bounds the rounding error of each Greek as computed. B_y bounds the difference from the *same graph* in exact arithmetic. The primitives are expanded into graph operations (Newton–Raphson steps, table lookups, series terms), each with $\epsilon_i = 2^{-(FL+1)}$; their approximation error is common to both evaluations and measured separately ($\le 1.1\times10^{-8}$ relative against exact functions). Each CORDIC unit is one operation whose error was measured against its exact integer model over 30,000 inputs: rotation within 1.14 ulp (modelled as 2 ulp); vectoring magnitude within 0.8 ulp; the vectoring angle within $1.75/|z|$ ulp, modelled as $2\max(1, 1/|z|)$ ulp. The COS truncation error is excluded (Section V-C). B_y takes 0.1 s per input in software, so it is a design-time tool for choosing and certifying the word length (Table VI), not attached to each result.

Generator and schedule. One Python description of setup, term ($N = 128$) and finish is an operation graph from which the generator derives the bit-accurate model, a double-precision model, the bound and a modulo schedule [30] emitted as Verilog (Fig. 1); the testbench compares every register against the model at the cycle it becomes valid. A

new term starts every initiation interval (II, Fig. 2); with 8 multipliers, 3 CORDIC rotators and 2 vectorers all three resources give II $= \lceil 250/8 \rceil = 32$, and the loop takes $127{\cdot}32{+}506{+}2 = 4{,}572$ cycles. The reverse sweep's extra 124 multiplications per term fit into multiplier slots the CORDIC-bound forward pass leaves idle, but only because the engine has twice the multipliers a price-only engine needs for the same II ($\lceil 126/32 \rceil = 4$).

IV. Implementation and Methodology

Configurations. `z7` (56-bit, 8 multipliers, all on chip, 4,733 cycles); `z7h` (the same with setup and finish on the host as bit-exact generated C, 128-term loop on the FPGA, 4,572 cycles); `zu` (64-bit, 32 multipliers, all on chip, 1,593 cycles). Each sits behind an AXI4-Lite register file, driven for testing by a JTAG-to-AXI master (Table II).

Inputs. A 21-case grid (moneyness, maturity, vol-of-vol, correlation; calls and puts); a 10,500-input sweep: 6,000 uniform over the domain (S_0=100, $K \in [60, 150]$, $T \in [0.1, 3]$, $r \in [0, 0.1]$, $v_0, \theta \in [0.005, 0.25]$, $\kappa \in [0.2, 6]$, $\xi \in [0.1, 1]$, $\rho \in [-0.95, 0.6]$), 500 in each of seven hard regimes (Feller violated, ρ near its limits, deep strikes, short and long maturities, high vol-of-vol, low variance) and 1,000 with one input outside; a held-out sweep of the same composition drawn after the shift ranges were fitted; 50 board cases; and 50 calibrations to NIFTY index options. Rounding error is measured against the same graph in double precision; the COS result separately against a Fourier-integral price.

Equal-throughput engines. Price-only (4 multipliers), adjoint (8) and factored forward-mode (10) `z7h`-style engines, II = 32, generated, checked bit-exact against their models and implemented out of context in Vivado 2025.2 (xc7z020clg484-1, 70 MHz). The forward datapath carries tangents without the per-term rescaling the adjoint applies, as a forward-mode AD tool would.

CPU baselines. The same algorithm in C++ (double, g++ 13) on an Intel i5-1155G7 laptop (4 cores): price only, bump-and-reprice, an optimised bump reusing the characteristic function, forward-mode AD, taped adjoint AD (CoDiPack [13]) and hand-derived analytic Greeks [12], `-O3 -march=native`; that build leaves the 128-term loop scalar, so a structure-of-arrays variant split into seven loops over the terms is built with `-Ofast -fopenmp-simd` and glibc's vector maths (a single-loop vectorised variant spills and is slower). All match the reference (vectorised to 9.3×10^{-12}). Latency: best of five passes over 2,000 inputs on one idle core; throughput: eight processes. Table III lists the assumptions.

TABLE II
Board implementations (Vivado, post-route, JTAG systems)

	ZedBoard	**ZCU102**	**ZCU104**
Device	xc7z020-1	xczu9eg-2	xczu7ev-2
Engine; cycles	`z7h`; 4,572	`zu`; 1,593	`zu`; 1,593
LUT	38,752 (72.8%)	77,515 (28.3%)	77,863 (33.8%)
FF	32,134 (30.2%)	63,547 (11.6%)	63,796 (13.8%)
DSP	72 (32.7%)	510 (20.2%)	510 (29.5%)
Clock (MHz)	70	100	100
WNS (ns)	+0.458	+1.601	+1.469
On silicon	yes	yes	no

Engine-alone $F_{\max}$ on the Zynq-7020 ≈79 MHz (56-bit carry chains). Totals include the JTAG-to-AXI master and interconnect.

TABLE III
Assumptions, rationale and checks

	Assumption	**Rationale; how checked**
A1	Bounded input domain	Shifters sized from measured ranges; held-out sweep: no in-domain flag, unflagged outside inputs within bound.
A2	$S_0 = 100$	Price homogeneous of degree one in (S_0, K); host rescales.
A3	N=128, range $c_1 \pm 10\sqrt{c_2}$	Standard [2]; method error measured separately (Sec. V-C).
A4	$[a, b]$ fixed when differentiating	A numerical parameter; reference uses the same; Greeks within method error of a Fourier integral.
A5	Calls by put–call parity	Call coefficients cancel in fixed point; worst call error 75× smaller.
A6	First-order bound	Second-order terms $\approx 2^{-FL}$ smaller; 50-digit check ≤0.24 of bound; holds at 44 bits.
A7	Local errors as in Sec. III	RTL bit-exact with the model; CORDIC errors measured.
A8	56/64-bit words	56 bits: shortest with grid Greek error $\leq 1.2 \times 10^{-5}$; held-out tail weaker (Table VI).
A9	Latency = cycles × period	JTAG cannot time the engine; bus transfers (39 words) and `z7h` host steps excluded.
A10	FPGA power from simulated activity	Routed netlist simulated on four bit-exact evaluations; vectorless 12.5% underestimated dynamic power 3–6×.
A11	CPU package power (HWiNFO)	WSL cannot read it; idle 5.5 W; power and throughput from the same run, Balanced plan, mains.
A12	First-order Greeks; vega $= \partial V/\partial v_0$	Gamma (second order) has a COS closed form [2]; not built.
A13	Hand-derived adjoint	Agrees with forward modes to 6×10^{-14}, CoDiPack to 1.3×10^{-10}.
A14	NIFTY: calibrated, $r = 0.06$	Realistic test inputs, all in the domain.

V. Results

A. Cost of the Greeks

On the same hardware the price and nine Greeks take 1.04× the price alone on the 56-bit design (4,733 vs. 4,568 cycles) and 1.55× on the 64-bit one, where the multipliers set the pace (Fig. 3). These figures flatter the adjoint, since a price-only engine reaches the same II with half the multipliers; at equal throughput the Greeks cost about 3% in cycles on both designs (4,733 vs. 4,613; 1,593 vs. 1,540). In Vivado (Table IV) they cost 1.03× the cycles, 1.68× the LUTs, 1.78× the flip-flops and 2× the DSP blocks of a right-sized price-only engine (hand-derived Greeks cost 1.5 pricings in time on the CPU). Most of the 11,901 added flip-flops are the tape: 297 more rotating 56-bit registers than the price-only schedule, part of which Vivado maps to shift-register LUTs. Bump-and-reprice takes 18.4× (56-bit) and 12.3× (64-bit) the adjoint's cycles on the same hardware; taking Delta and $\partial V/\partial K$ from the COS sum and bumping only the other seven inputs (15 pricings) would be about 14.5× (estimated).

B. Adjoint against forward mode

Three forward-mode datapaths were built by the same generator: *dense* (all directions through every operation, as a vector forward-mode tool; a weak baseline), *sparse* (structural zeros skipped) and *factored* (exponent derivatives formed once

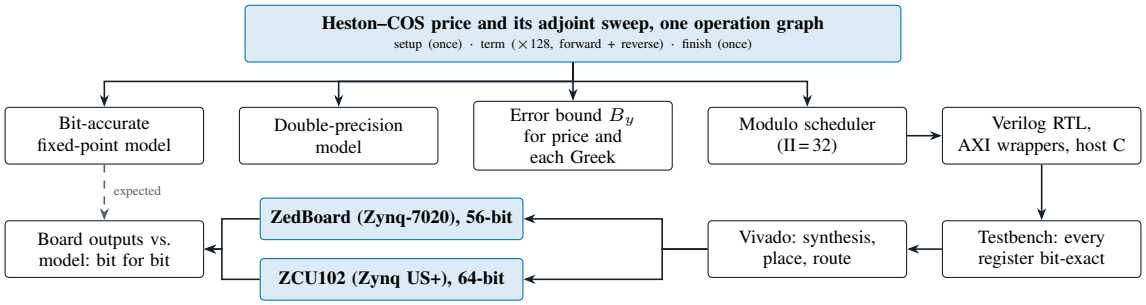


Fig. 1. Design flow. One operation graph yields the bit-accurate model, the error bound and the scheduled RTL; the RTL is checked register by register against the model, implemented in Vivado and run on two boards, whose outputs are compared bit for bit with the same model.

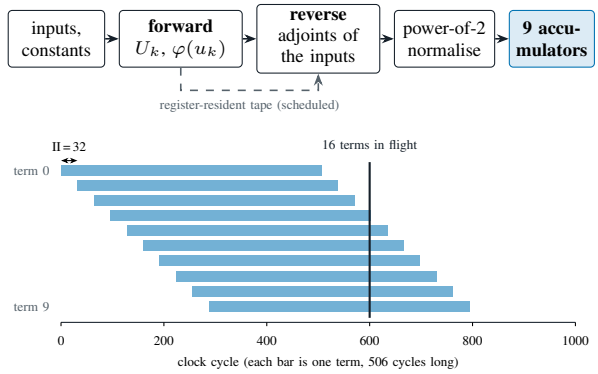


Fig. 2. Top: one COS term; the reverse sweep shares the forward datapath and reads forward values from scheduled rotating registers. Bottom: the modulo schedule (first 10 of 128 terms).

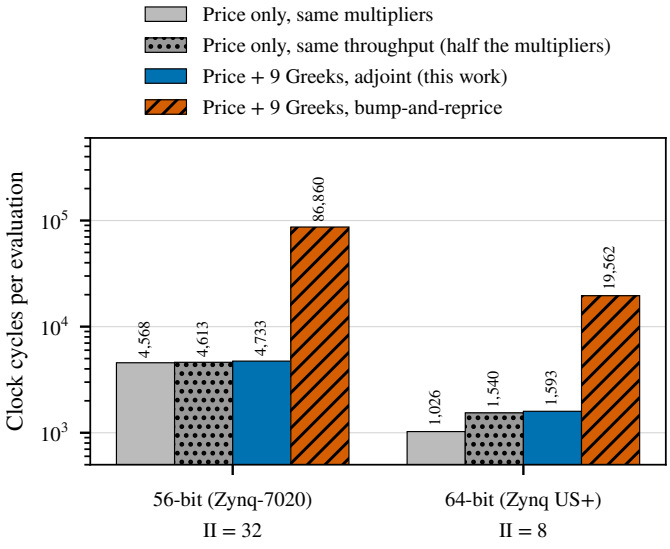


Fig. 3. Cycles per evaluation (cycle-exact RTL simulation): price only on the same hardware and at the same II with half the multipliers, the adjoint engine, and bump-and-reprice.

per term, as a hand derivation of analytic Greeks). All match the adjoint to 6×10^{-14} in double precision. Per term the adjoint needs 250 multiplications against 313, 396 and 802

TABLE IV
ENGINES OF EQUAL THROUGHPUT (II = 32), ZYNQ-7020, 70 MHZ

Engine	Mult.	LUT	FF	DSP	Cycles	Worst err.
Price only	4	21,251	15,303	36	4,457	—
Adjoint	**8**	**35,782**	**27,204**	**72**	4,572	$\mathbf{1.2 \times 10^{-5}}$
Fwd., factored	10	38,199	30,514	90	4,449	7.4×10^{-5}

Out of context, all bit-exact against their models (WNS $+0.377$, $+0.109$, $+0.275$ ns). Worst err.: worst relative Greek error over the 21-case grid.

TABLE V
CYCLES BY DATAPATH AND SHARED MULTIPLIERS (SCHEDULER)

		56-bit		64-bit	
Datapath	**Mult./term**	**8**	**16**	**16**	**32**
Price only	126	4,566	4,561	1,538	1,024
Adjoint	**250**	**4,731**	4,619	**2,612**	**1,591**
Forward, factored	313	5,622	4,571	3,069	1,795
Forward, sparse	396	6,968	4,571	3,690	2,171
Forward, dense	802	13,597	7,150	7,155	3,822

(Table V); with 16 multipliers all but the dense one are CORDIC-bound and the factored one is slightly faster. At equal throughput (Table IV) the factored engine is 2.7% faster in cycles but needs 6.8% more LUTs, 12.2% more flip-flops and 25% more DSP blocks, and is 6× less accurate at the same word length because its tangents are not rescaled. The adjoint's advantage over a well-factored forward datapath is real but modest.

C. Accuracy and the bound

Rounding. On the 21-case grid (Fig. 4) the 56-bit price is accurate to 6.1×10^{-7} and the Greeks to at most 1.2×10^{-5}; the 64-bit design gains one to two orders of magnitude; no output exceeds 0.103 of its bound. Over the 10,500-input sweep no unflagged result exceeded its bound (worst 0.378) and nothing overflowed; on the held-out sweep no in-domain input was flagged and the worst in-domain ratio was 0.356. In 50-digit arithmetic on 100 held-out inputs the error is at most 0.24 of the bound, and the bound holds at 44 bits, where second-order terms are 4,096× larger (A6). The grid understates the tail (Table VI): the price and market Greeks keep at least 5.3 significant figures for 99% of inputs, but

TABLE VI
HELD-OUT SWEEP, 56-BIT DESIGN, 9,500 IN-DOMAIN INPUTS: SIGNIFICANT FIGURES AND THE BOUND RELATIVE TO THE VALUE

	Significant figures			$B_y/\lvert y\rvert$		
Output	**Med.**	**P99**	**Worst**	**Med.**	**P99**	**Max**
V	7.6	5.6	4.4	1e-6	6e-5	6e-4
∂_{S_0}	8.1	5.7	3.9	3e-7	5e-5	3e-3
∂_K	7.9	5.7	3.9	7e-7	6e-5	3e-3
∂_T	7.6	5.5	4.0	1e-6	9e-5	3e-3
∂_r	7.9	5.9	4.4	6e-7	4e-5	7e-4
∂_{v_0}	7.6	5.3	3.3	8e-7	2e-4	4e-2
∂_κ	6.0	3.5	2.2	3e-5	4e-3	7e-2
∂_θ	7.2	4.9	3.5	2e-6	3e-4	6e-3
∂_ξ	5.9	3.0	1.4	4e-5	2e-2	1.1
∂_ρ	7.0	4.9	3.7	4e-6	3e-4	6e-3

Against the same graph in double precision; inputs where a Greek is within 1% of zero excluded. A sample of the domain, not a supremum.

TABLE VII
SILICON VALIDATION (OUTPUTS COMPARED BIT FOR BIT WITH THE MODEL)

Board	**Inputs**	**Cases**	**Bit-exact**	**Err./bound**
ZedBoard	vector + sweep	1 + 50	459 / 459	0.199
ZCU102	vector + sweep	1 + 50	510 / 510	0.243
ZCU102	NIFTY calibrations	50	500 / 500	0.255
Total			**1,469 / 1,469**	≤ 0.255

ZedBoard: 9 sums per case, finished on the host by bit-exact code; ZCU102: 10 outputs computed on chip. No `range_err`.

$\partial V/\partial\kappa$ and $\partial V/\partial\xi$ keep 3.5 and 3.0, and 2.2 and 1.4 at worst; for $\partial V/\partial\xi$ the bound can exceed the value. Against bump-and-reprice on the same pricer (Fig. 5) the adjoint is 32–560× more accurate (median 145×) even with the best bump size chosen afterwards per Greek.

COS method error. The bound covers rounding only. Against a Fourier-integral price on the 50 board cases the method error has a median of 2×10^{-11} for the price but reaches 7.8×10^{-3} for vega at $\xi = 1$, $T = 0.34$ (128 terms are too few; 256 remove it), far above the bounded rounding error, and the engine does not flag it. A deployment would restrict the domain or use $N = 256$, roughly doubling the cycles.

D. Silicon validation

Cases ran back to back with no reset (Table VII). Every output matched the bit-accurate model, so the accuracy results and the bound hold on silicon; the worst error/bound ratio there was 0.255 and the worst 64-bit price error 7.3×10^{-7}.

E. Against a CPU

The FPGA latencies are computed (A9): they exclude the bus transfers and, for the ZedBoard, the host's setup and finish; the CPU figures are measured whole evaluations (Fig. 6, Table VIII). The ZCU102 engine is 4.9× faster than forward-mode AD, 6.5× than taped AAD, 15× than bump-and-reprice and 1.3× than scalar analytic Greeks, but one core running the vectorised analytic Greeks (13.6 μs) is 1.17× faster than the engine before its transfers are counted, and four cores give 216,000 evaluations per second against 62,800 for one engine (about 188,000 for the three that would fit on the ZCU102 by LUTs, not built). We claim no speed advantage.

TABLE VIII
LATENCY, THROUGHPUT AND ENERGY PER EVALUATION

Method	**μs**	**Cost**	**Eval./s**	**Energy**
CPU, price only	13.7	1×	—	—
CPU, bump-and-reprice	238.1	17.4×	—	—
CPU, optimised bump	146.6	10.7×	—	—
CPU, adjoint (CoDiPack)	103.8	7.6×	26,900[a]	—
CPU, forward mode	78.0	5.7×	34,000[a]	—
CPU, analytic Greeks	20.3	1.5×	158,000[a]	54.6 μJ[b]
CPU, analytic, vectorised[e]	13.6	—	216,000[a]	40.9 μJ[b]
FPGA, ZedBoard, 70 MHz (loop)[d]	65.3	1.04×	15,300	44 μJ[c]
FPGA, ZCU102, 100 MHz[d]	15.9	1.55×	62,800	40 μJ[c]

Cost: time relative to the price alone on the same platform and hardware. [a]4 cores, 8 processes. [b]Package power × run time / evaluations, 8 processes. [c]Chip level, simulated activity (A10). [d]Computed; transfers excluded. [e]Structure of arrays, 29 Oct (scalar 22.7 μs in the same session); throughput from the energy run.

Energy, counted at chip level on both sides: the FPGA device's on-chip power with the engine as its only logic (dynamic from simulated activity, plus the device's static power), 0.676 W and 2.485 W, gives 44 and 40 μJ per evaluation; the CPU package power (HWiNFO, A11) is 7.9 and 8.8 W with eight processes of the scalar and vectorised methods, giving 54.6 and 40.9 μJ (167 and 115 μJ on one core). Neither includes DRAM or the board, and the FPGA excludes the host and bus. The 64-bit engine thus matches the vectorised CPU and uses 1.4× less than the scalar build; counting only power above idle (CPU, 15.4 μJ) against dynamic power (FPGA, 30 μJ), the CPU is ahead. We claim no energy advantage.

VI. DISCUSSION AND CONCLUSION

For a pricer without data-dependent control flow, a hand-derived reverse sweep can share one statically scheduled datapath with the forward pass, its tape in scheduled registers. At equal throughput this costs about 1.7× the area of a price-only engine at 1.03× its cycles, and is cheaper in multipliers and logic, and more accurate at the same word length, than a well-factored forward-mode datapath built by the same flow; each Greek has a first-order rounding bound that held on every input tested and on silicon, on two FPGA families and on market calibrations. These are the claims. Speed and energy against a CPU are not: a vectorised laptop core is faster, four cores give more throughput, and the energy per evaluation is the same at chip level.

Limitations. The adjoint's advantage over a factored forward datapath is modest; Delta and $\partial V/\partial K$ are nearly free in any COS implementation; the bound is a design-time tool that excludes the COS method error, which dominates in a few in-domain cases; Gamma is not computed; latency was computed, not measured, and board-level power is unmeasured; the Zynq-7020 design closes at 79 MHz; the domain is bounded. Strike chains, in which one reverse sweep per term serves every strike of an expiry (32 strikes in 4,892 cycles, scheduled but not built), help a CPU as much (2.3 μs per strike). Deriving the

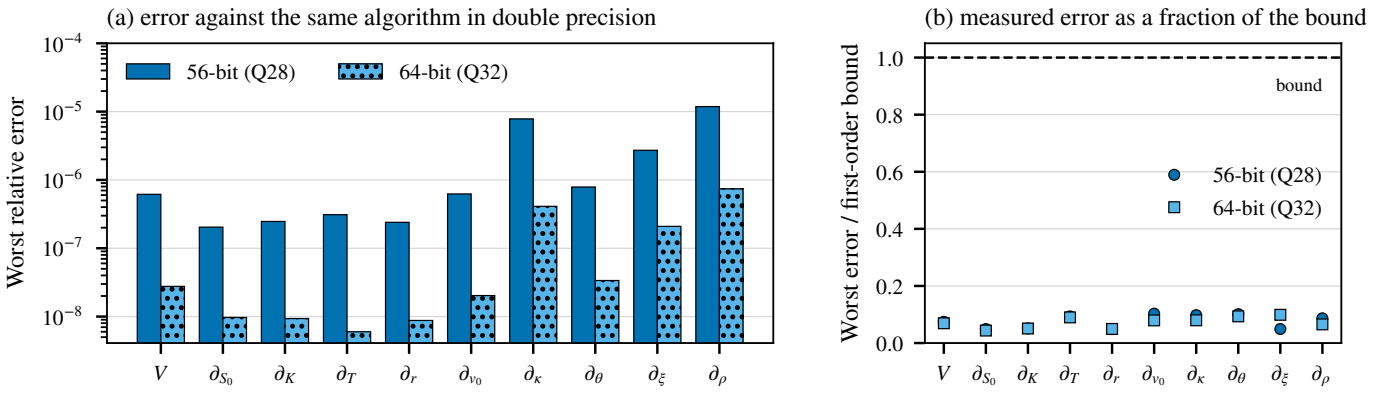


Fig. 4. Rounding accuracy over the 21-case grid. (a) Worst relative error of the price V and each Greek against the same graph in double precision. (b) Worst error divided by the bound (2).

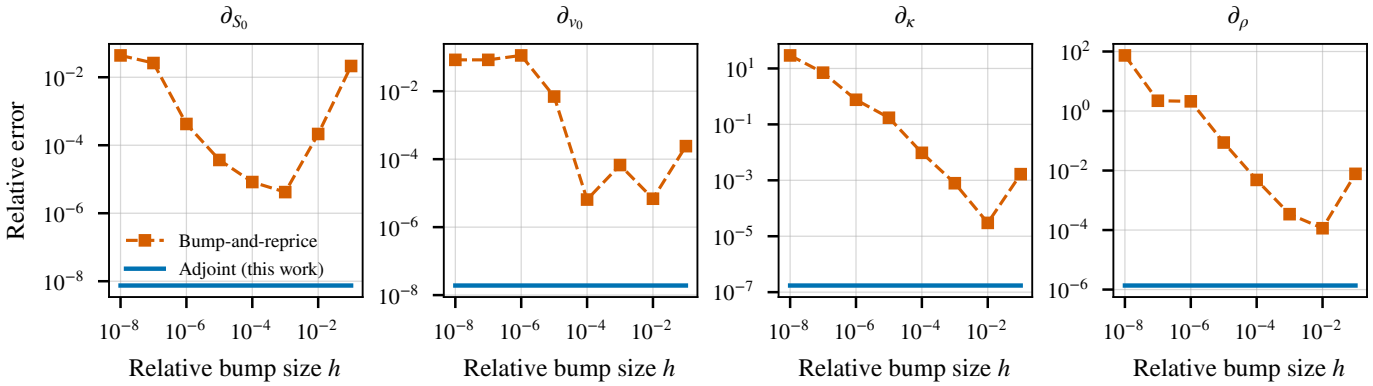


Fig. 5. Relative error of bump-and-reprice against relative bump size, and of the adjoint engine, for four Greeks (56-bit design, base case); the other five behave alike.

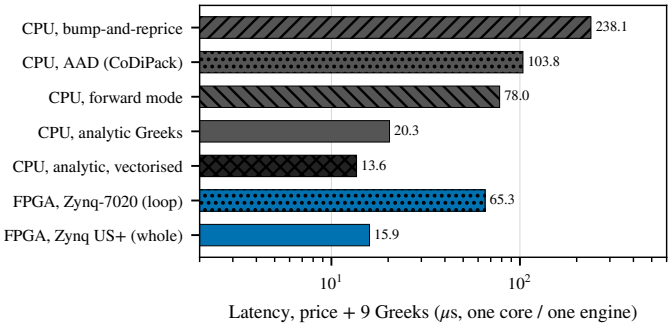


Fig. 6. Latency of one price and nine Greeks: one CPU core (i5-1155G7, idle; measured) and one FPGA engine (cycles × period, transfers excluded).

reverse sweep automatically, adding Gamma, measuring end-to-end latency and board power, and placing several engines per device are the next steps.

References

[1] S. L. Heston, "A closed-form solution for options with stochastic volatility with applications to bond and currency options," *Rev. Financial Stud.*, vol. 6, no. 2, pp. 327–343, 1993.

[2] F. Fang and C. W. Oosterlee, "A novel pricing method for European options based on Fourier-cosine series expansions," *SIAM J. Sci. Comput.*, vol. 31, no. 2, pp. 826–848, 2008.

[3] G. Junike, "On the number of terms in the COS method for European option pricing," *Numer. Math.*, 2024, doi:10.1007/s00211-024-01402-1.

[4] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert, "The little Heston trap," *Wilmott Mag.*, pp. 83–92, Jan. 2007.

[5] M. Giles and P. Glasserman, "Smoking adjoints: Fast Monte Carlo Greeks," *Risk*, vol. 19, pp. 88–92, Jan. 2006.

[6] L. Capriotti, "Fast Greeks by algorithmic differentiation," *J. Comput. Finance*, vol. 14, no. 3, pp. 3–35, 2011.

[7] L. Capriotti and M. Giles, "15 years of adjoint algorithmic differentiation (AAD) in finance," *Quant. Finance*, vol. 24, no. 9, pp. 1353–1379, 2024.

[8] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA, USA: SIAM, 2008.

[9] A. Griewank, "Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation," *Optim. Methods Softw.*, vol. 1, no. 1, pp. 35–54, 1992.

[10] L. Hascoët and V. Pascual, "The Tapenade automatic differentiation tool: Principles, model, and specification," *ACM Trans. Math. Softw.*, vol. 39, no. 3, 2013.

[11] S. Linnainmaa, "Taylor expansion of the accumulated rounding error," *BIT Numer. Math.*, vol. 16, no. 2, pp. 146–160, 1976.

[12] Y. Cui, S. del Baño Rollin, and G. Germano, "Full and fast calibration of the Heston stochastic volatility model," *Eur. J. Oper. Res.*, vol. 263, no. 2, pp. 625–638, 2017.

[13] M. Sagebaum, T. Albring, and N. R. Gauger, "High-performance derivative computations using CoDiPack," *ACM Trans. Math. Softw.*, vol. 45, no. 4, 2019.

[14] M. Klaisoongnoen, N. Brown, and O. Thomson Brown, "Low-power option Greeks: Efficiency-driven market risk analysis using FPGAs," in *Proc. Int. Symp. Highly-Efficient Accelerators and Reconfigurable Technologies (HEART)*, 2022, pp. 95–101.

[15] M. Klaisoongnoen, N. Brown, and O. Thomson Brown, "Fast and energy-efficient derivatives risk analysis: Streaming option Greeks on Xilinx and Intel FPGAs," in *Proc. Int. Workshop Heterogeneous High-Performance Reconfigurable Computing (H2RC), SC22*, 2022.

[16] M. Klaisoongnoen, N. Brown, T. Dykes, J. R. Jones, and U.-U. Haus, "Evaluating Versal AI Engines for option price discovery in market risk analysis," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2024.

[17] C. de Schryver *et al.*, "An energy efficient FPGA accelerator for Monte Carlo option pricing with the Heston model," in *Proc. Int. Conf. Reconfigurable Computing and FPGAs (ReConFig)*, 2011.

[18] C. de Schryver, P. Torruella, and N. Wehn, "A multi-level Monte Carlo FPGA accelerator for option pricing in the Heston model," in *Proc. Design, Automation & Test in Europe (DATE)*, 2013, pp. 248–253.

[19] Q. Jin, D. B. Thomas, W. Luk, and B. Cope, "Exploring reconfigurable architectures for tree-based option pricing models," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 2, no. 4, art. 21, 2009.

[20] A. H. T. Tse, D. Thomas, and W. Luk, "Design exploration of quadrature methods in option pricing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 20, no. 5, pp. 818–826, 2012.

[21] AMD, "Vitis Libraries: Quantitative Finance Library," [Online]. Available: https://github.com/Xilinx/Vitis_Libraries

[22] B. Zhang and C. W. Oosterlee, "Option pricing with COS method on graphics processing units," in *Proc. IEEE Int. Symp. Parallel & Distributed Processing (IPDPS)*, 2009, pp. 1–8.

[23] B. Zhang and C. W. Oosterlee, "Acceleration of option pricing technique on graphics processing units," *Concurrency Computat.: Pract. Exper.*, vol. 26, no. 9, pp. 1626–1639, 2014.

[24] F. Gremse, A. Höfter, L. Razik, F. Kiessling, and U. Naumann, "GPU-accelerated adjoint algorithmic differentiation," *Comput. Phys. Commun.*, vol. 200, pp. 300–311, 2016.

[25] A. A. Gaffar, O. Mencer, W. Luk, P. Y. K. Cheung, and N. Shirazi, "Floating-point bitwidth analysis via automatic differentiation," in *Proc. IEEE Int. Conf. Field-Programmable Technology (FPT)*, 2002, pp. 158–165.

[26] D.-U. Lee, A. A. Gaffar, R. C. C. Cheung, O. Mencer, W. Luk, and G. A. Constantinides, "Accuracy-guaranteed bit-width optimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 25, no. 10, pp. 1990–2000, 2006.

[27] F. de Dinechin, C. Lauter, and G. Melquiond, "Certifying the floating-point implementation of an elementary function using Gappa," *IEEE Trans. Comput.*, vol. 60, no. 2, pp. 242–253, 2011.

[28] P. T. P. Tang, "Table-driven implementation of the exponential function in IEEE floating-point arithmetic," *ACM Trans. Math. Softw.*, vol. 15, no. 2, pp. 144–157, 1989.

[29] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, 1959.

[30] B. R. Rau, "Iterative modulo scheduling: An algorithm for software pipelining loops," in *Proc. 27th Annu. Int. Symp. Microarchitecture (MICRO)*, 1994, pp. 63–74.